Movie Night, Part 2: Bringing the Board to Life with State

A step-by-step tutorial on state, events, and the difference between props and state

At the end of Part 1 your Movie Night board looked finished, but nothing on it worked. Click **+** and the rating stays put. In this session you make every control do something: the ratings change, movies can be removed from the list, the WATCHED badges toggle, and the header and footer counts keep up on their own.

To get there you need one new idea: **state**, the data a component remembers and React watches. The session follows the same path as the scoreboard examples in the Module 5 code folder: a class component, its state, events, `setState()`, binding `this`, updating from the previous state, application state, removing items, and finally the `useState` hook.

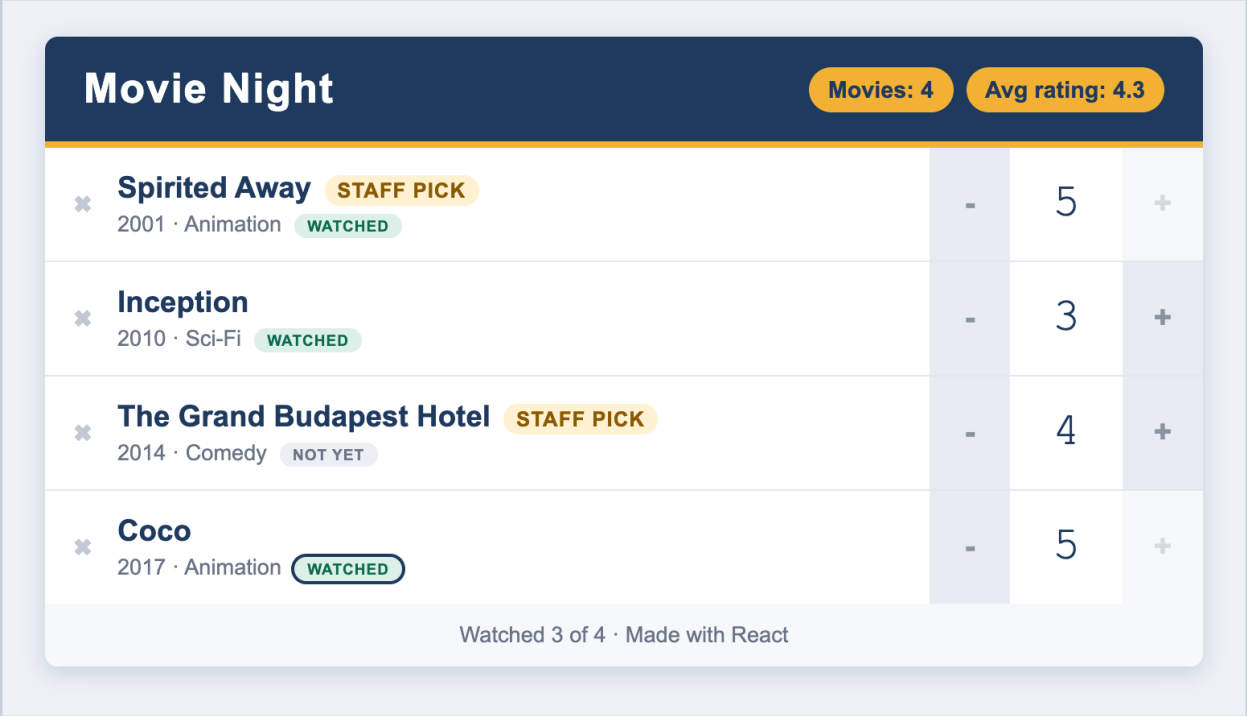


Figure 1 — the finished board: every button works, and the counts update by themselves

> **HOW TO USE THIS HANDOUT**
> Same rules as Part 1: your instructor works through each step at the front of the room, and you type the code yourself rather than pasting it. Every step ends with a **Checkpoint** folder in `code/checkpoints/`. If you fall behind, copy that folder's `app.js` over yours and carry on.

CONTENTS

Step	Topic	Minutes
00	Pick up where Part 1 ended	5
01	Why props are not enough	10
02	Write a component as a class	10
03	Give the component state	10
04	Handle a click	10
05	Update state with `setState()`	10
06	Bind event handlers to the component	10
07	Update state based on the previous state	10
08	Props versus state	8
09	Create the application state	12
10	Remove items from state	14
11	Lift state up: one source of truth for ratings	14
12	Your turn: toggle WATCHED	10
13	The same app with the `useState` hook	10
14	Wrap-up and self-check	7
	Total	**150**

After the tutorial you will do a 30-minute **Exercise Lab** (a separate handout), which you submit for one mark.

Step 00 — Pick up where Part 1 ended

Goal: a working copy of the finished Part 1 app, with the Part 2 stylesheet.

1. Use your own `movie-night` folder from Part 1. If you don't have it, copy the folder `code/starter/`, which holds the finished Part 1 app.
2. Replace your `app.css` with the one in `code/starter/`. It is Part 1's stylesheet with a few additions at the bottom: a ✖ remove button, clickable badges, greyed-out buttons and an "empty list" message. As before, you do not edit it.
3. Open `index.html` with **Live Server**, open the **Console**, and install or enable **React Developer Tools** if it is not already on. You will use its **Components** tab all session.

> **CHECKPOINT 00**
> `code/starter/`. Your page should look exactly like Figure 1 of Part 1.

Step 01 — Why props are not enough

Goal: see for yourself why a component needs something more than props and ordinary variables.

Click **+** on any row. Nothing happens, and it shouldn't. Look at `Rating`:

```
const Rating = (props) => {
  return (
    <div className="rating">
      <button className="rating-action decrement"> - </button>
      <span className="rating-score">{ props.score }</span>
      <button className="rating-action increment"> + </button>
    </div>
  );
};
```

The score comes from `props.score`. In Part 1 you learned that props are **read-only**: the parent owns them, and `props.score = 6` throws an error. So `Rating` cannot change its own score through props.

What about an ordinary variable? Try it.

1. Replace `Rating` with this experiment. `onClick` runs a function when the button is clicked; Step 04 covers events properly.

```
// An experiment: can an ordinary variable remember the score?
const Rating = (props) => {
  let score = props.score;

  const increment = () => {
    score = score + 1;
    console.log('score is now', score);
  };

  return (
    <div className="rating">
      <button className="rating-action decrement"> - </button>
      <span className="rating-score">{ score }</span>
      <button className="rating-action increment" onClick={increment}> + </button>
    </div>
  );
};
```

2. Click **+** on *Spirited Away* three times and watch both the page and the Console.

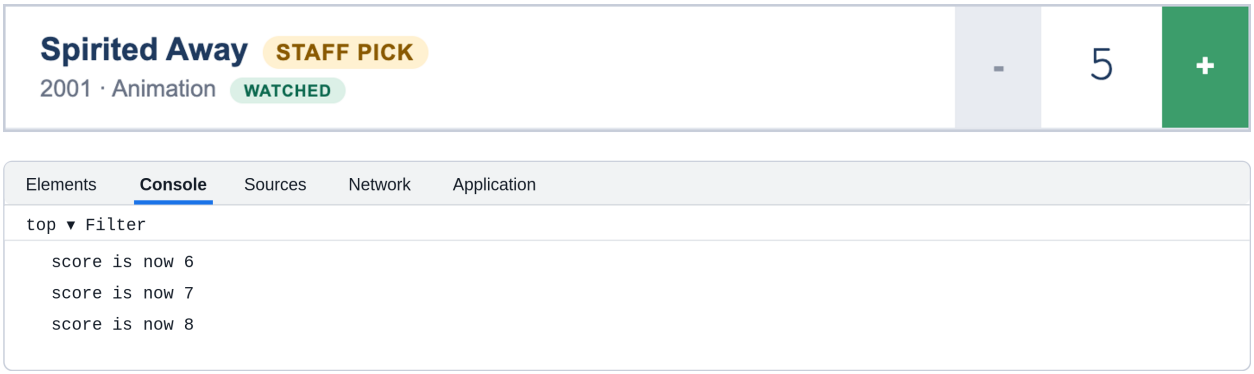


Figure 2 — the variable changes, but the page does not

The Console proves the variable really changes (6, 7, 8), but the page still shows **5**. Two things go wrong:

Problem	Why
React is never told.	React draws a component by *calling* its function. Changing a local variable does not make React call `Rating` again, so the old output stays on screen.
The variable would not survive anyway.	Even if React did call `Rating` again, the first line, `let score = props.score`, would run again and reset the score to 5. A local variable is created fresh on every call.

A component needs data that **survives between renders** and that **React watches**, so that changing it redraws the component. That is exactly what **state** is.

> **CHECKPOINT 01**
> `code/checkpoints/step-01-plain-variable/`. Keep this experiment in your file. You replace `Rating` again in the next step.

Step 02 — Write a component as a class

Goal: rewrite `Rating` as a **class component**, the original way React components hold state.

Until 2019, the only way to give a component state was to write it as a JavaScript **class**. You will meet class components in plenty of existing code, and they make the parts of state very visible, so we learn state with them first. Step 13 shows the newer function version.

1. Replace the experiment with a class:

```
class Rating extends React.Component {
  render() {
    return (
      <div className="rating">
        <button className="rating-action decrement"> - </button>
        <span className="rating-score">{ this.props.score }</span>
        <button className="rating-action increment"> + </button>
      </div>
    );
  }
}
```

2. Save. The page looks exactly as it did at the end of Part 1.

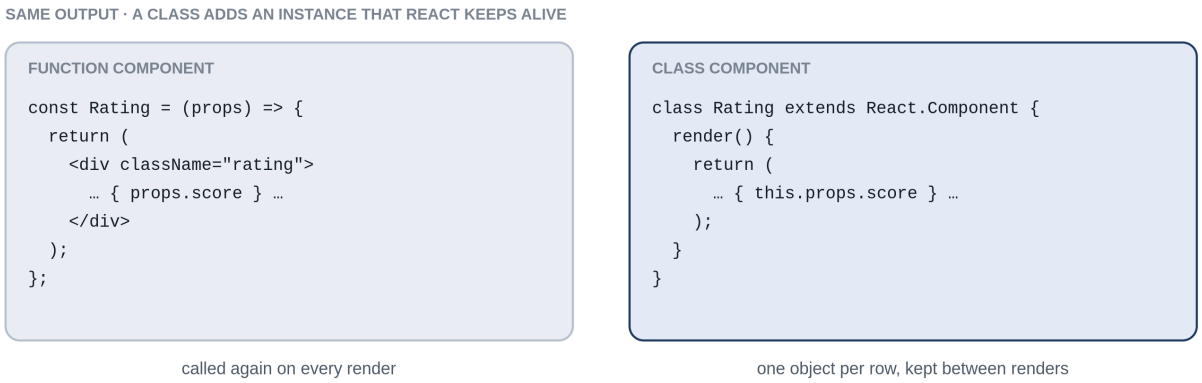


Figure 3 — the same component written two ways

In a function component	In a class component
`const Rating = (props) => { … }`	`class Rating extends React.Component { … }`
The function body *is* the render	A method named `render()` that returns the JSX. It is required.
`props.score`	`this.props.score`: React stores the props on the instance
Called again on every render	One **instance** is created and kept while the component is on the page

That last row is the key to state. React creates one `Rating` object per row and keeps it alive. Anything stored on that object survives between renders, which is what the local variable in Step 01 could not do.

> CHECKPOINT 02
> `code/checkpoints/step-02-class-component/`

Step 03 — Give the component state

Goal: let each `Rating` own its own score.

State is an object that holds the data a component manages itself. In a class you create it as a property named `state`.

1. Add the `state` property at the top of the class, and display `this.state.score` instead of `this.props.score`:

```
class Rating extends React.Component {
  state = {
    score: this.props.score
  };

  render() {
    return (
      <div className="rating">
        <button className="rating-action decrement"> - </button>
        <span className="rating-score">{ this.state.score }</span>
        <button className="rating-action increment"> + </button>
      </div>
    );
  }
}
```

The page looks the same, but there is a real difference. `Rating` uses the prop only **once**, as the *starting* value. From now on the score belongs to `Rating`.

2. Open **Components** in the developer tools and click one of the `Rating` components. The right-hand pane now has two sections, **props** and **state**.

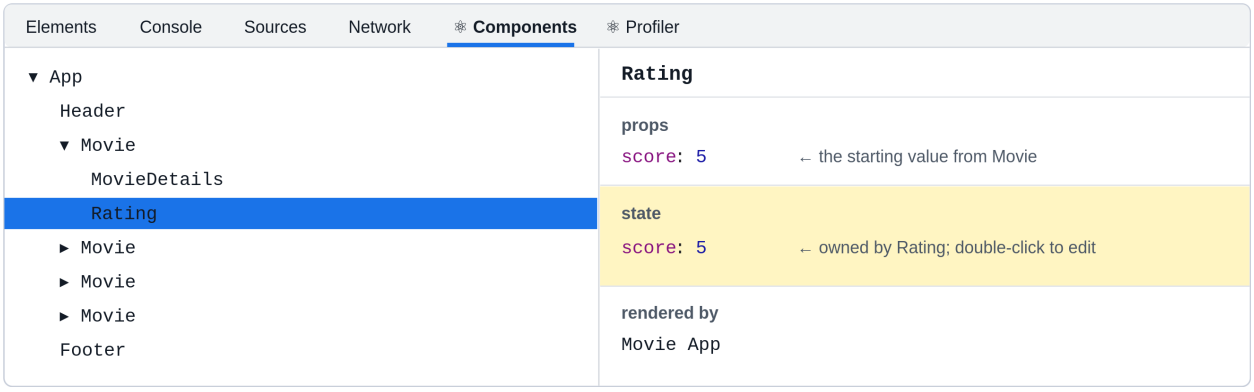


Figure 4 — React DevTools shows the component's props and its state separately

3. In the DevTools **state** section, double-click the score, type `2` and press Enter. That row on the page changes to **2**, and only that row. Each `Rating` instance has its own state. Then edit a value under **props** and notice that the page does *not* change, because the component no longer displays its prop.

> **NAMING TIP**
> When a prop is only used as a starting value for state, a good name makes that clear, for example `initialScore`. We keep `score` to avoid changing `Movie`, but Step 09 uses exactly this convention: `initialMovies` is copied into state once.

> **CHECKPOINT 03**
> `code/checkpoints/step-03-stateful-component/`

Step 04 — Handle a click

Goal: run your own code when a button is clicked.

React events look like HTML's `onclick` attribute, with two differences:

HTML	React
`onclick` (lowercase)	`onClick` (camelCase), and likewise `onChange`, `onSubmit`, `onMouseEnter`
A string of code: `onclick="increment()"`	A **function** in braces: `onClick={this.incrementScore}`

1. Add a method to the class, above `render()`, and connect it to the **+** button:

```
  incrementScore() {
    console.log('Hi, from inside incrementScore!');
  }
```

```
        <button className="rating-action increment" onClick={this.incrementScore}> +
</button>
```

2. Save and click **+** a few times. Each click prints the message.

Notice that `onClick={this.incrementScore}` has **no parentheses**. You are handing React the function so that *React* can call it later, when the click happens.

3. Try the most common mistake: add the parentheses, `onClick={this.incrementScore()}`, save, and reload **without clicking anything**.

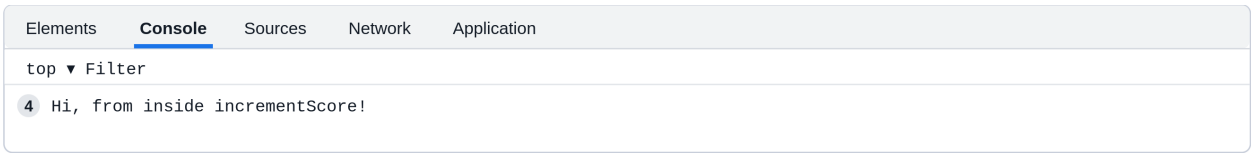


Figure 5 — with parentheses, the function runs during rendering, once per row

The message prints **four times** as soon as the page loads (once for each `Rating`), and clicking does nothing. With parentheses, JavaScript calls `incrementScore()` *while building the JSX*, and hands its return value (`undefined`) to `onClick`. Remove the parentheses again.

> **EVENT HANDLER NAMES**
> React developers usually name the method after what it does (`incrementScore`), or prefix it with `handle` (`handleClick`). When a handler is passed as a prop, the prop is often prefixed with `on` (`onRemove`). This handout keeps the names used in the course's scoreboard code.

> **CHECKPOINT 04**
> `code/checkpoints/step-04-handling-events/`

Step 05 — Update state with setState()

Goal: change the state, and let React redraw the component.

You **never** assign to `this.state` directly. You call **`this.setState()`** with an object describing what should change. `setState()` does two things: it updates the state, and it tells React to **re-render** the component, which calls `render()` again with the new value.

1. Before using it, see what happens if you skip it. Temporarily make `incrementScore` change the state directly:

```
incrementScore() {
  this.state.score = this.state.score + 1;    // WRONG: React is not told
}
```

Click **+**. Nothing changes on the page. It is Step 01 again: the value changes, but React does not know.

2. Now do it properly. Replace the body with a `setState()` call:

```
incrementScore() {
  this.setState({
    score: this.state.score + 1
  });
}
```

3. Save and click **+**. The page still does not change, and the Console shows a red error:

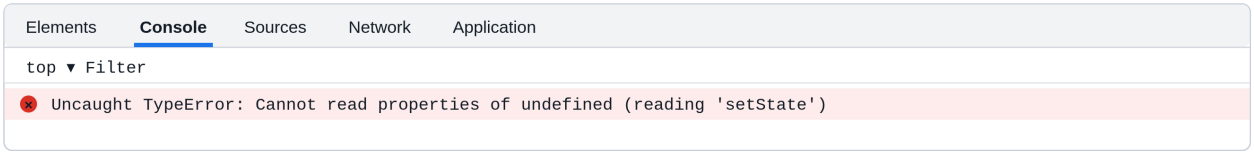


Figure 6 — inside the handler, this is undefined

`this.setState` failed because `this` is `undefined`. React calls your method as a **plain function** when the click happens. It does not call it as `rating.incrementScore()`, so inside the method `this` no longer refers to the component. Step 06 fixes that.

> **CHECKPOINT 05**
> `code/checkpoints/step-05-broken-this/` reproduces this error on purpose. There is nothing to fix yet. Go on to Step 06.

Step 06 — Bind event handlers to the component

Goal: make sure `this` inside a handler always means the component.

Connecting a method to its component is called **binding**. You will see three ways of doing it:

Technique	What it looks like
Bind in the constructor	`constructor(props) { super(props); this.incrementScore = this.incrementScore.bind(this); }`
Arrow function in the JSX	`onClick={() => this.incrementScore()}`
Arrow function as a class property (the one we use)	`incrementScore = () => { … }`

An arrow function does not have its own `this`. It uses the `this` of the place where it was written. Written as a class property, that place is the component instance, so `this` is always right, however React calls it.

1. Convert `incrementScore` into an arrow-function property, and add a matching `decrementScore`:

```
  incrementScore = () => {
    this.setState({
      score: this.state.score + 1
    });
  }

  decrementScore = () => {
    this.setState({
      score: this.state.score - 1
    });
  }
```

2. Connect **–** as well:

```
        <button className="rating-action decrement" onClick={this.decrementScore}> -
</button>
        <span className="rating-score">{ this.state.score }</span>
        <button className="rating-action increment" onClick={this.incrementScore}> +
</button>
```

3. Save. Click **+** twice on *Spirited Away*, and **–** once on *Inception*.

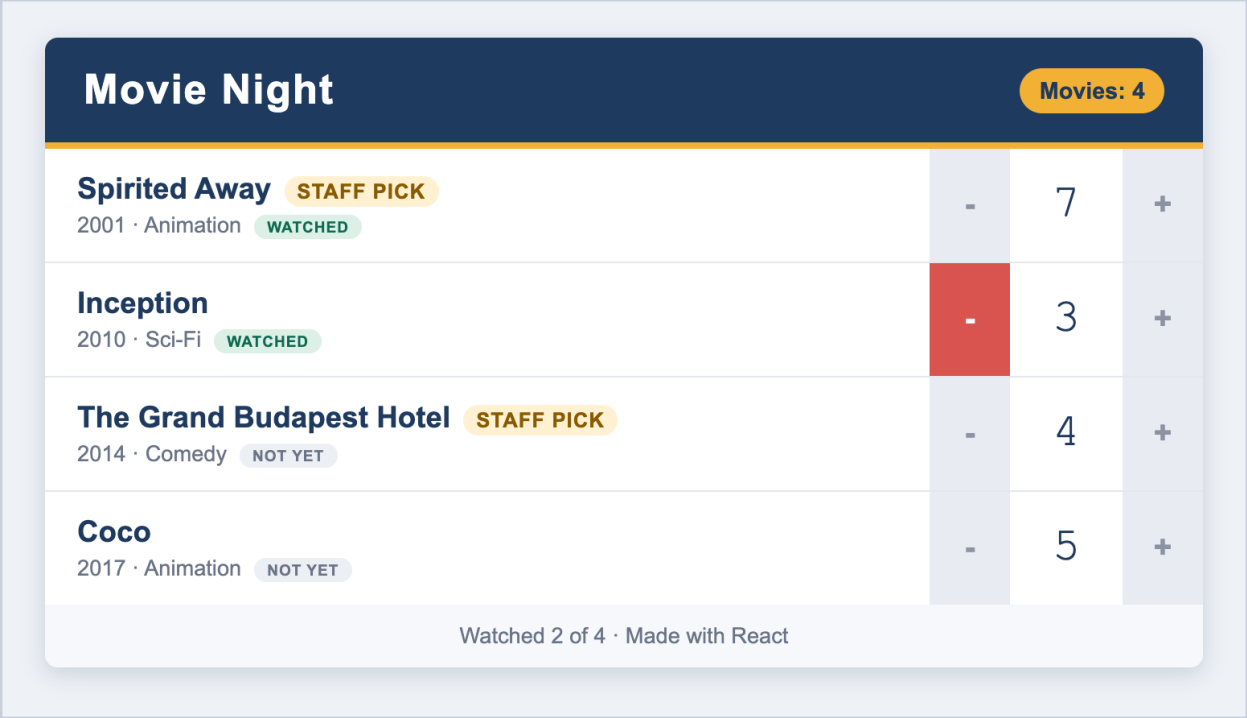


Figure 7 — the buttons work, and each row keeps its own score; Spirited Away has climbed to 7

Every click calls `setState()`, React re-renders that `Rating`, and the new score appears. Each row has its own instance, so the rows do not affect each other.

There is a bug, though: ratings are meant to run from 0 to 5, and *Spirited Away* is now on **7**. You fix it in the next step.

> `CHECKPOINT 06`
> `code/checkpoints/step-06-bind-handlers/`

Step 07 — Update state based on the previous state

Goal: write state updates that are always correct, and keep the score between 0 and 5.

React may **batch** several `setState()` calls together and apply them later, for speed. So while your handler runs, `this.state` may hold a value that is about to be replaced. Prove it:

1. Temporarily make `incrementScore` call `setState()` **twice**:

```
incrementScore = () => {
  this.setState({ score: this.state.score + 1 });
  this.setState({ score: this.state.score + 1 });
}
```

Click **+** on *Inception* (4). You might expect 6, but you get **5**. Both calls read the *same* `this.state.score` (4), and both ask for 5.

2. The fix is to pass `setState()` a **function** instead of an object. React calls the function with the latest state, conventionally named `prevState`, and uses the object it returns:

```
incrementScore = () => {
  this.setState( prevState => ({
    score: prevState.score + 1
  }));
  this.setState( prevState => ({
    score: prevState.score + 1
  }));
}
```

Click **+** on *Inception* again: 4 becomes **6**. The second function received the result of the first.

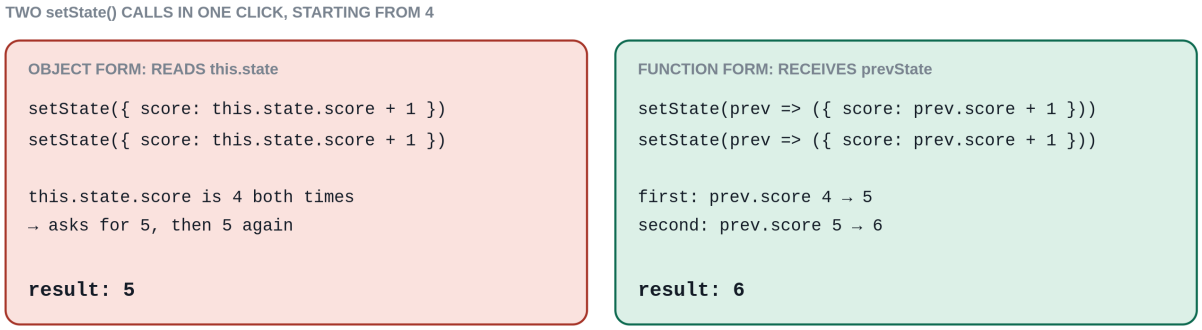


Figure 8 — an object reads a value that may be stale; a function receives the latest state

> **THE RULE**
> Whenever the new state is calculated *from* the old state (adding, toggling, filtering a list), use the function form: `this.setState(prevState => ({ … }))`. Note the parentheses around the object. Without them, `{` would open a function body.

3. Now write the real handlers: one update each, kept between 0 and 5 with `Math.min` and `Math.max`:

```
incrementScore = () => {
  this.setState( prevState => ({
    score: Math.min(prevState.score + 1, 5)
  }));
}

decrementScore = () => {
  this.setState( prevState => ({
    score: Math.max(prevState.score - 1, 0)
  }));
}
```

4. Grey out a button when it can't do anything. `disabled` takes a true/false expression:

```
render() {
  return (
    <div className="rating">
      <button
        className="rating-action decrement"
        onClick={this.decrementScore}
        disabled={this.state.score === 0}
      > - </button>
      <span className="rating-score">{ this.state.score }</span>
      <button
        className="rating-action increment"
        onClick={this.incrementScore}
        disabled={this.state.score === 5}
      > + </button>
    </div>
  );
}
```

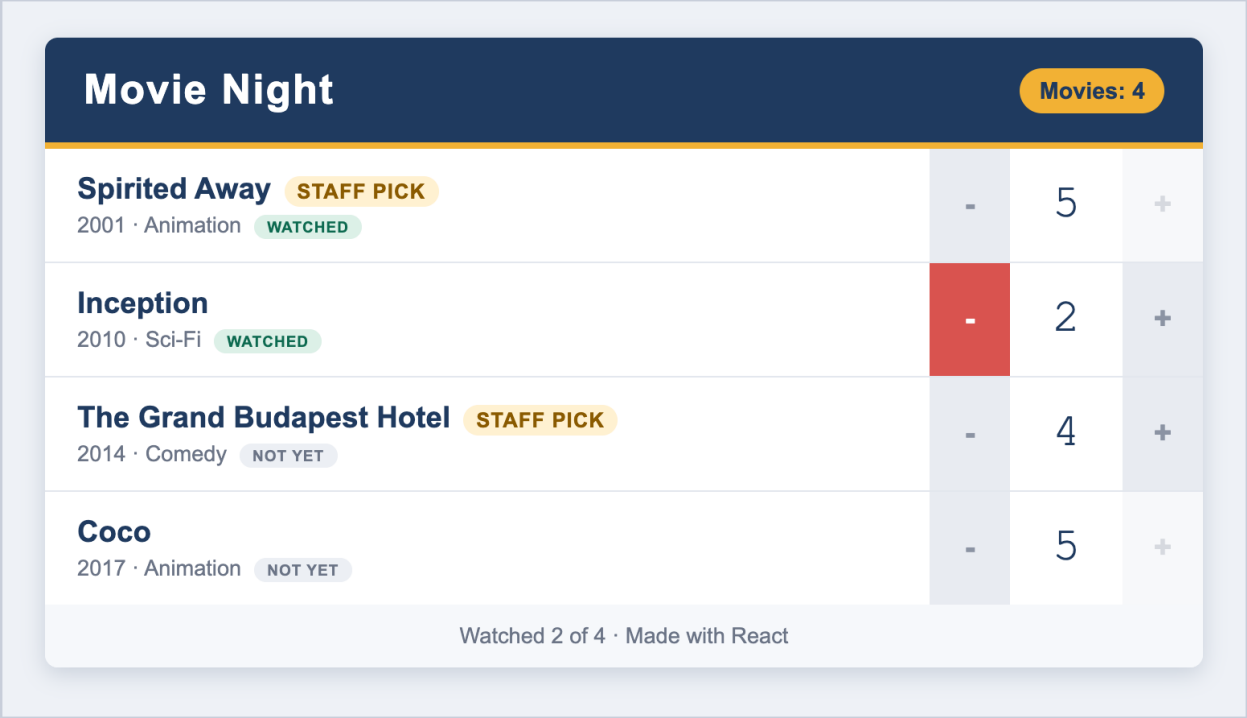


Figure 9 — scores stay between 0 and 5; + is greyed out at 5

> CHECKPOINT 07
> `code/checkpoints/step-07-previous-state/`

Step 08 — Props versus state

Goal: be able to say, for any piece of data, whether it should be a prop or state.

You now have both kinds of data in one component. `Rating` receives `score` as a **prop** and keeps its own `score` in **state**. Here is how they compare:

	Props	State
Where it comes from	Passed in by the **parent**	Created and owned **inside** the component
Who can change it	Only the parent. Read-only to the child.	Only the component itself, through `setState()`
How you read it	`props.x` / `this.props.x`	`this.state.x` (or a hook variable, Step 13)

	Props	State
Changing it re-renders?	Yes. When the parent passes new props, the child re-renders.	Yes. `setState()` re-renders the component and its children.
Its purpose	**Configure** a component from outside	**Remember** things that change while the app runs
Analogy	A function's **arguments**	A function's private **memory**

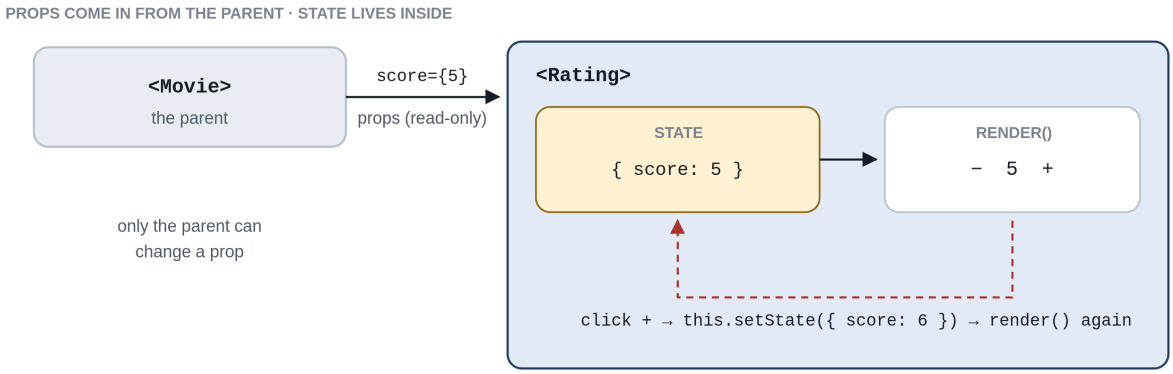


Figure 10 — props flow in from the parent; state lives inside and changes with setState()

They work together: **one component's state becomes its children's props.** When state changes, React re-renders that component, and the children receive the new values as props.

Should it be state? Three questions

Ask these about any piece of data, in order. If any answer is **yes**, it is **not** state:

1. Is it passed in from a parent? Then it is a **prop**.
2. Does it stay the same for as long as the app runs? Then it is a **constant** (like the `title`).
3. Can you calculate it from other props or state? Then **compute it in `render()`** (like the watched count in `Footer`).

Only data that changes over time, and that cannot be worked out from anything else, belongs in state.

Try it (3 min). Classify each of these as prop, state, constant or computed: the `"Movie Night"` title; the list of movies (once movies can be removed); the number of movies shown in the header; whether a single movie has been watched; the average rating. Check your answers with the person next to you. Steps 09 to 12 give the answers away.

Step 09 — Create the application state

Goal: move the list of movies into state, in the component at the top of the tree.

Soon you will remove movies and change their ratings. When that happens, three components must agree: the **Header** (movie count), the **list**, and the **Footer** (watched count). Data that several components share should live in their **closest common parent**, which here is `App`. This is called **application state**.

1. Convert `App` into a class with a `state` that starts from the `initialMovies` prop:

```
class App extends React.Component {
  state = {
    movies: this.props.initialMovies
  };

  render() {
    const watchedCount = this.state.movies.filter(movie => movie.watched).length;

    return (
      <div className="watchlist">
        <Header
          title="Movie Night"
          totalMovies={this.state.movies.length}
        />

        {/* Movie list */}
        {this.state.movies.map( movie =>
          <Movie
            title={movie.title}
            year={movie.year}
            genre={movie.genre}
            rating={movie.rating}
            watched={movie.watched}
            staffPick={movie.staffPick}
            key={movie.id.toString()}
          />
        )}

        <Footer
          watchedCount={watchedCount}
          totalMovies={this.state.movies.length}
        />
      </div>
    );
  }
}
```

Every `props.initialMovies` became `this.state.movies`. The render line at the bottom of the file, `root.render(<App initialMovies={movies} />)`, stays the same.

2. Save. The page looks the same. Check **Components**: `App` now shows a **state** section holding an array of four movies.

Notice what is **not** in state. `watchedCount` and `totalMovies` can be *computed* from the movies, so `render()` works them out each time. If you stored them in state as well, you would have to keep two copies in sync, and sooner or later they would disagree.

> **CHECKPOINT 09**
> `code/checkpoints/step-09-application-state/`

Step 10 — Remove items from state

Goal: a ✖ button on each row that removes that movie, and the counts update by themselves.

This step needs a new pattern. The ✖ button lives in `Movie`, but the list of movies lives in `App`'s state, and **only `App` can change its state**. The solution:

Direction	What travels	How
Down (App → Movie)	A function that knows how to remove a movie	A prop: `removeMovie={this.handleRemoveMovie}`
Up (Movie → App)	"Remove *this* id"	The child **calls** the prop: `props.removeMovie(props.id)`

Data flows down as props. **Events flow up** as calls to functions that came down as props.

1. In `App`, add the handler above `render()`. `filter()` builds a **new** array without the movie whose id matches:

```
handleRemoveMovie = (id) => {
  this.setState( prevState => ({
    movies: prevState.movies.filter( movie => movie.id !== id )
  }));
}
```

2. In `App`'s `map()`, pass each movie its `id` and the handler. Add these two lines next to `key`:

```
            id={movie.id}
            removeMovie={this.handleRemoveMovie}
```

You need a separate `id` prop because, as you saw in Part 1, `key` is reserved by React and cannot be read as `props.key`.

3. In `Movie`, add the button as the first child of `<div className="movie">`:

```
      <button className="remove-movie" onClick={() =>
props.removeMovie(props.id)}>✖</button>
```

This time the handler **must** be wrapped in an arrow function. `onClick={props.removeMovie}` would call it with the click event instead of the id, and `onClick={props.removeMovie(props.id)}` would call it during rendering (the mistake from Step 04): every movie vanishes as soon as the page loads, and React logs `Cannot update during an existing state transition`. `() => props.removeMovie(props.id)` creates a small function that React calls on click, and that function calls the handler with the right argument.

4. Save and click ✖ on *Inception*.

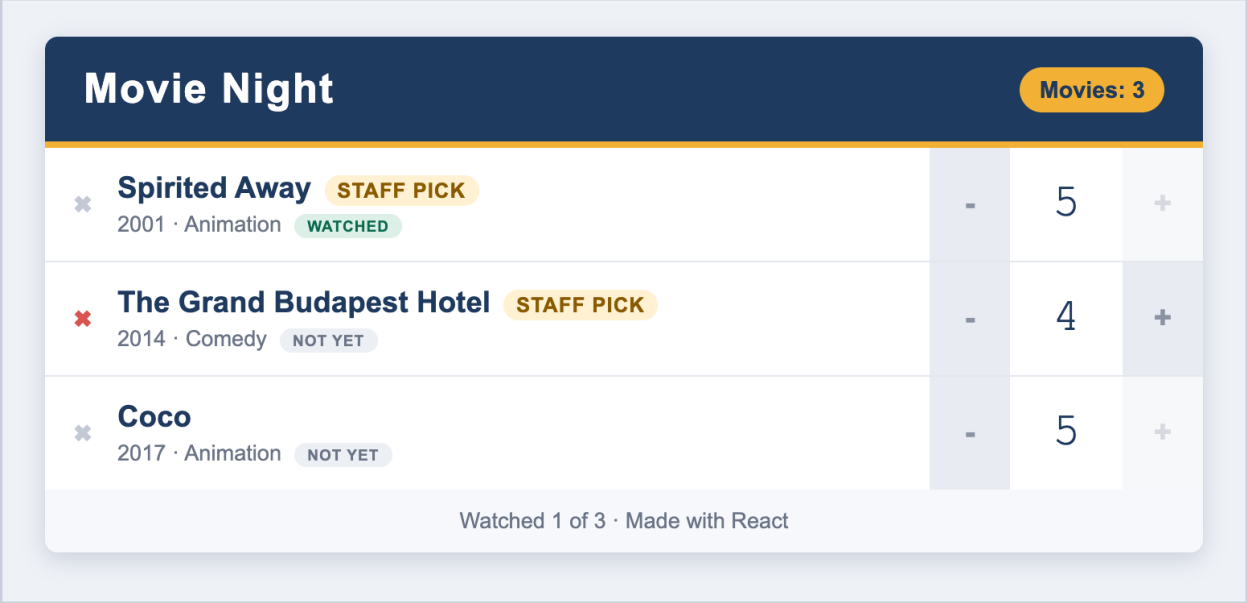


Figure 11 — one setState() in App: the row disappears, and Movies and Watched both drop by one

You only changed the array in `App`'s state. React re-rendered `App`, and the **Header**, the **list** and the **Footer** all received new props computed from that one array.

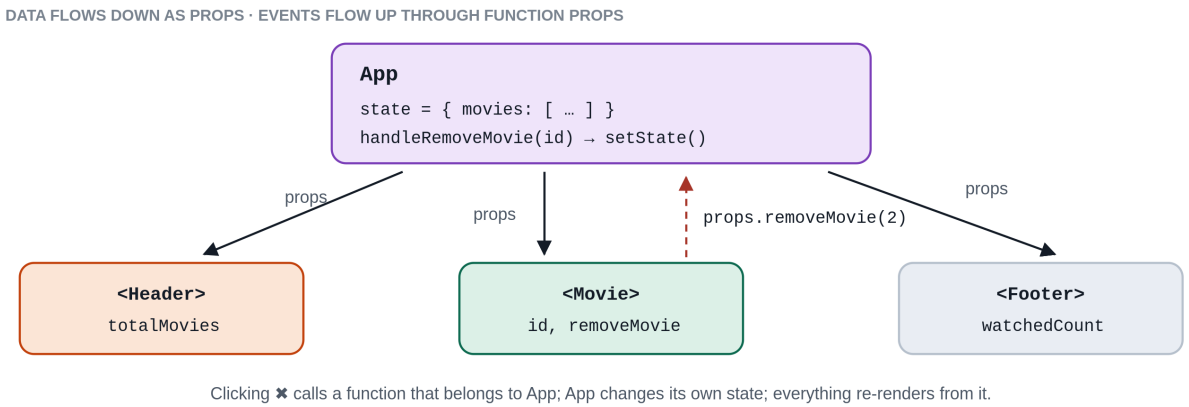


Figure 12 — data flows down as props; the remove request flows up as a function call

5. Remove all four movies. You get an empty card. Add a friendly message just above the `map()` in `App`, using `&&` from Part 1:

```
      {this.state.movies.length === 0 &&
        <p className="empty">The watchlist is empty. Time to pick a movie!</p>}
```

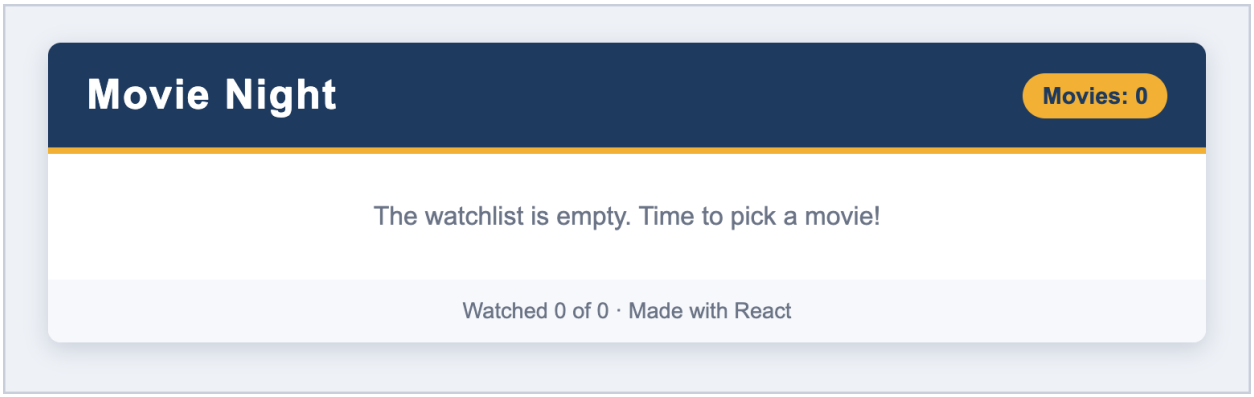


Figure 13 — every movie removed: the && message appears

Reload the page to get the four movies back. State lives in memory, so a reload starts again from `initialMovies`.

> **NEVER CHANGE STATE IN PLACE**
> `prevState.movies.splice(index, 1)` or `.push(…)` would change the existing array, and React could miss the change. Always build a **new** array with `filter()`, `map()` or the spread operator `[...]`, and hand it to `setState()`.

> **CHECKPOINT 10**
> `code/checkpoints/step-10-remove-items/`

Step 11 — Lift state up: one source of truth for ratings

Goal: show the **average rating** in the header, and make it update when a rating changes.

1. Think before typing. The header needs the ratings, but each rating is hidden inside its own `Rating` instance's state. `App` cannot read a child's state, and data only flows **down**. There is also a second problem: `App`'s state still holds the *original* ratings, so there are now two versions of every rating, and they disagree as soon as you click.

The fix is called **lifting state up**: delete the state from `Rating` and keep the ratings in one place only, `App`'s `movies` array. `Rating` goes back to being a simple function that displays a prop and reports clicks upward, just as `Movie` does with ✖.

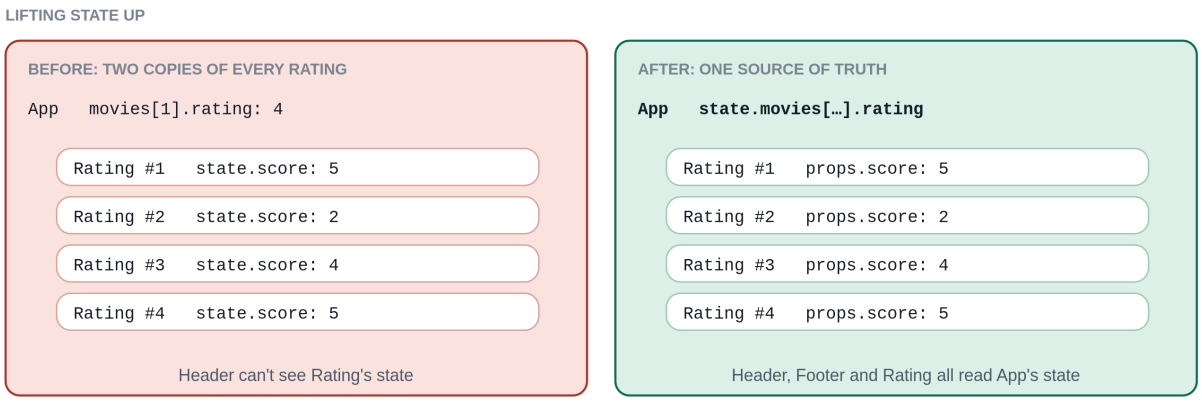


Figure 14 — before: every Rating owns a hidden copy; after: App owns the only copy

2. Replace the whole `Rating` class with a function component:

```
// No state of its own any more: the score comes from App, and clicks are reported
to App
const Rating = (props) => {
  return (
    <div className="rating">
      <button
        className="rating-action decrement"
        onClick={() => props.changeRating(-1, props.id)}
        disabled={props.score === 0}
      > - </button>
      <span className="rating-score">{ props.score }</span>
      <button
        className="rating-action increment"
        onClick={() => props.changeRating(1, props.id)}
        disabled={props.score === 5}
      > + </button>
    </div>
  );
};
```

3. `Movie` passes down what `Rating` needs. Replace `<Rating score={props.rating} />` with:

```
      <Rating
        score={props.rating}
        id={props.id}
        changeRating={props.changeRating}
      />
```

4. In `App`, add a handler that changes **one** movie's rating. `map()` returns a new array: the movie whose id matches is replaced by a **copy** with a new rating (`{ ...movie, rating: … }`), and every other movie is passed through unchanged:

```
  handleRatingChange = (delta, id) => {
    this.setState( prevState => ({
      movies: prevState.movies.map( movie =>
        movie.id === id
          ? { ...movie, rating: Math.min(Math.max(movie.rating + delta, 0), 5) }
          : movie
      )
    }));
  }
```

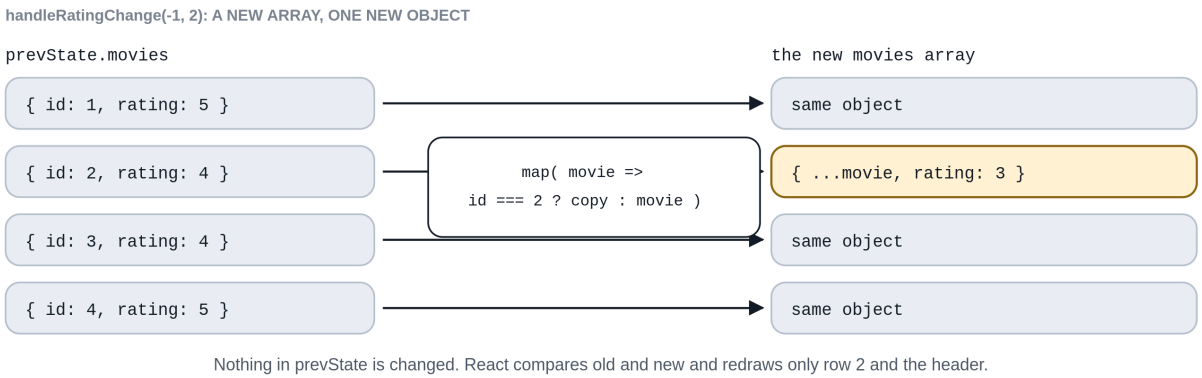


Figure 15 — updating one item: map() copies the array, the spread copies the one object that changes

5. Pass it to every `Movie` in the `map()`: `changeRating={this.handleRatingChange}`.
6. Compute the average at the top of `render()`, with `reduce()` adding up the ratings:

```
    const watchedCount = this.state.movies.filter(movie => movie.watched).length;
    const totalStars = this.state.movies.reduce((sum, movie) => sum + movie.rating,
0);
    const averageRating = this.state.movies.length > 0
      ? (totalStars / this.state.movies.length).toFixed(1)
      : '–';
```

The ternary avoids dividing by zero when the list is empty.

7. Pass it to the header, `averageRating={averageRating}`, and display it in `Header` with a second pill:

```
      <span className="stats">Avg rating: { props.averageRating }</span>
```

8. Save. Lower *Inception* twice and raise *The Grand Budapest Hotel* once. The average follows every click.

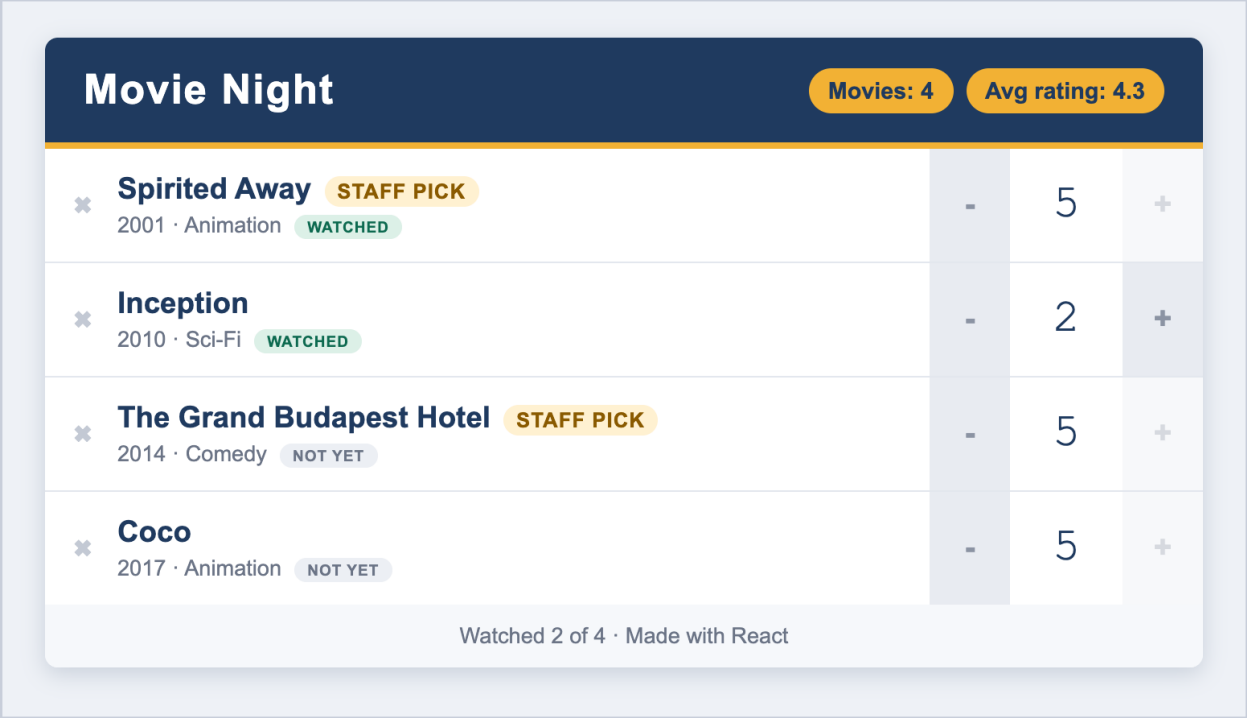


Figure 16 — the ratings live in App now, so the header can show their average

```
CHECKPOINT 11
code/checkpoints/step-11-lift-state-up/
```

Step 12 — Your turn: toggle WATCHED

Goal: apply the same pattern on your own. Clicking a movie's WATCHED / NOT YET badge flips it, and the footer count follows.

You need exactly the three pieces from Step 11:

1. A handler in `App`, `handleToggleWatched(id)`, that uses `setState` with `prevState` and `map()`, and copies the matching movie with `watched` flipped (`!movie.watched`).
2. Props that carry the handler and the id down: `App` → `Movie` → `MovieDetails`.
3. In `MovieDetails`, a `<button>` instead of the badge `<span>` that calls the handler on click. The stylesheet already styles `button.badge`.

Try it for 6 minutes before reading on.

Solution. The handler in `App`:

```
  handleToggleWatched = (id) => {
    this.setState( prevState => ({
      movies: prevState.movies.map( movie =>
        movie.id === id
          ? { ...movie, watched: !movie.watched }
          : movie
      )
    }));
  }
```

In `App`'s `map()`, add `toggleWatched={this.handleToggleWatched}`. In `Movie`, add two props to `<MovieDetails … />`:

```
          id={props.id}
          toggleWatched={props.toggleWatched}
```

In `MovieDetails`, replace the WATCHED/NOT YET ternary with one button whose class and text both depend on `props.watched`:

```
        <button
          className={ props.watched ? "badge badge-watched" : "badge badge-
unwatched" }
          onClick={() => props.toggleWatched(props.id)}
        >
          { props.watched ? "WATCHED" : "NOT YET" }
        </button>
```

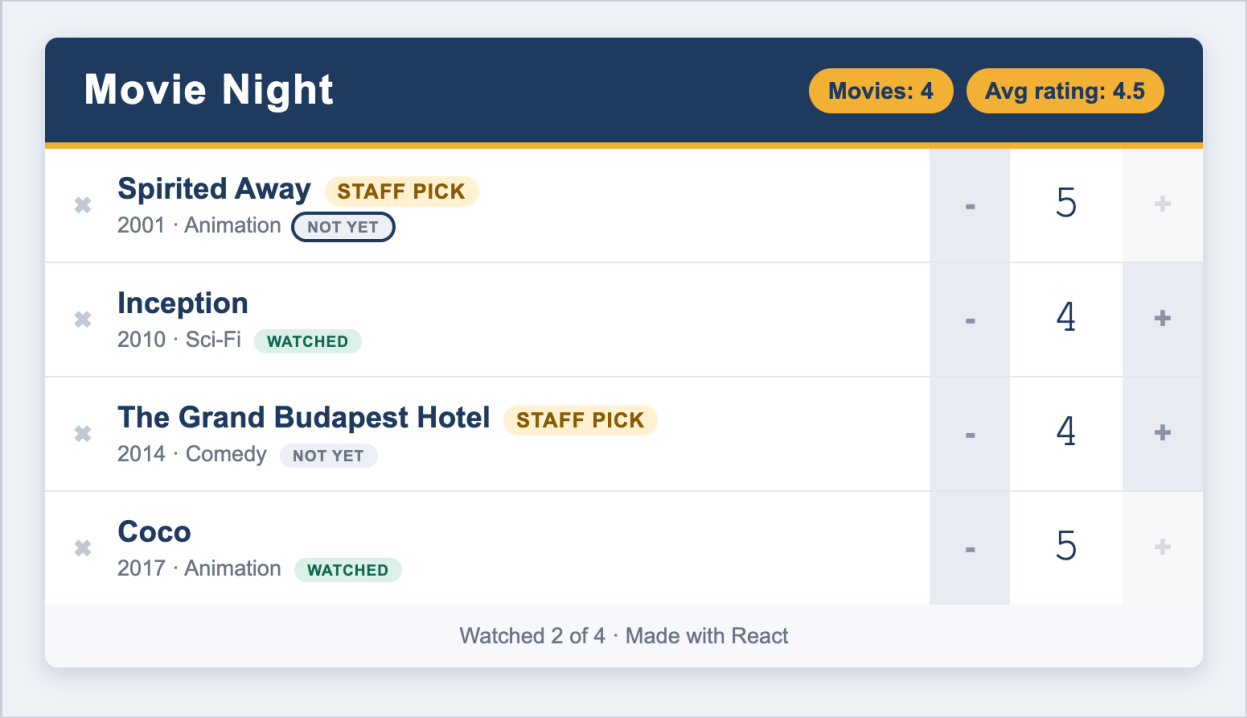


Figure 17 — Coco toggled to WATCHED and Spirited Away back to NOT YET; the footer still says 2 of 4

`MovieDetails` never knew the list existed. It only received a function and called it. That is what makes these components reusable.

> **CHECKPOINT 12**
> `code/checkpoints/step-12-toggle-watched/`

Step 13 — The same app with the useState hook

Goal: see how the same state looks in a function component, which is how most React code is written today.

Since React 16.8, function components can have state through **hooks**. Module 6 covers hooks in depth. Today you only need one, `useState`, and you can see that every idea from this session carries over unchanged.

1. At the very top of `app.js`, add:

```
const { useState } = React;
```

(With a build tool such as Vite you would write `import { useState } from 'react';` instead.)

2. Replace the `App` class with this function:

```
const App = (props) => {
  const [movies, setMovies] = useState(props.initialMovies);

  const handleRemoveMovie = (id) => {
    setMovies( prevMovies => prevMovies.filter( movie => movie.id !== id ) );
  };

  const handleRatingChange = (delta, id) => {
    setMovies( prevMovies => prevMovies.map( movie =>
      movie.id === id
        ? { ...movie, rating: Math.min(Math.max(movie.rating + delta, 0), 5) }
        : movie
    ));
  };

  const handleToggleWatched = (id) => {
    setMovies( prevMovies => prevMovies.map( movie =>
      movie.id === id
        ? { ...movie, watched: !movie.watched }
        : movie
    ));
  };

  const watchedCount = movies.filter(movie => movie.watched).length;
  const totalStars = movies.reduce((sum, movie) => sum + movie.rating, 0);
  const averageRating = movies.length > 0
    ? (totalStars / movies.length).toFixed(1)
    : '-';

  return (
    <div className="watchlist">
      <Header
        title="Movie Night"
        totalMovies={movies.length}
        averageRating={averageRating}
      />

      {/* Movie list */}
      {movies.length === 0 &&
        <p className="empty">The watchlist is empty. Time to pick a movie!</p>}

      {movies.map( movie =>
        <Movie
          title={movie.title}
          year={movie.year}
          genre={movie.genre}
          rating={movie.rating}
          watched={movie.watched}
          staffPick={movie.staffPick}
          id={movie.id}
          key={movie.id.toString()}
```

```
          removeMovie={handleRemoveMovie}
          changeRating={handleRatingChange}
          toggleWatched={handleToggleWatched}
        />
      )}

      <Footer
        watchedCount={watchedCount}
        totalMovies={movies.length}
      />
    </div>
  );
};
```

3. Save and test every button. The app behaves exactly as before.

Class component	Function component with `useState`
`state = { movies: this.props.initialMovies };`	`const [movies, setMovies] = useState(props.initialMovies);`
`this.state.movies`	`movies`
`this.setState(prevState => ({ movies: … }))`	`setMovies(prevMovies => …)`
Handlers are arrow-function class properties, to keep `this`	Handlers are ordinary `const` functions: no `this`, nothing to bind
`render() { … return (…); }`	The function body is the render

`useState(initialValue)` returns a pair, which we unpack with array destructuring: the current value, and a function that replaces it and re-renders. Note one difference: `setState` in a class **merges** the object you give it into the existing state, while a `useState` setter **replaces** the value completely.

> **CHECKPOINT 13 — THE FINISHED APP**
> `code/checkpoints/step-13-use-state-final/`

Step 14 — Wrap-up and self-check

What you built, and the idea behind each part

You wrote	The idea
`let score` that changed but never showed	Ordinary variables are not watched by React and reset on every render
`class Rating extends React.Component { render() {…} }`	A class component keeps one instance alive, so it can remember things
`state = { score: … }` and `this.state.score`	State is data a component owns and that changes over time
`onClick={this.incrementScore}`	Event props take a function; no parentheses, or it runs during render
`this.setState({ … })`	The only way to change state; it re-renders the component
`incrementScore = () => { … }`	Arrow-function class properties keep `this` bound to the component
`setState(prevState => ({ … }))`	Calculate new state from the latest state, never from a possibly stale `this.state`
`App` with `state = { movies }`	Shared data lives in the closest common parent
`removeMovie={this.handleRemoveMovie}` / `props.removeMovie(id)`	Data flows down as props; events flow up through function props
`filter()`, `map()` + `{ ...movie }`	Update arrays and objects by building new ones, never in place
Header average, footer count	Values derived from state are computed in render, not stored
`const [movies, setMovies] = useState(…)`	Hooks give function components the same state

Self-check

1. In Step 01 the variable reached 8 but the page showed 5. Give both reasons.
2. What is the difference between `this.props.score` and `this.state.score` inside `Rating` in Step 03? Which one does the page show?

3. Why does `onClick={this.incrementScore()}` print the message when the page loads, and not when you click?
4. Why was `this` undefined in Step 05, and why does `incrementScore = () => { … }` fix it?
5. Two calls of `this.setState({ score: this.state.score + 1 })` add only 1. Why? What should you write instead?
6. Name three differences between props and state.
7. Why does the list of movies live in `App` and not in `Movie`?
8. `Movie` cannot change `App`'s state, yet clicking ✖ in a `Movie` removes it. How?
9. Why is `averageRating` computed in `render()` instead of being kept in state?
10. What is wrong with `prevState.movies.push(newMovie)` inside `setState()`?

Answer key

1. React was never told that anything changed, so it did not re-render, and even if it had, `let score = props.score` would have run again and reset the value to 5.
2. `this.props.score` is the value the parent passed in, and here it is only used as the starting value. `this.state.score` is the component's own value, which changes with `setState()`. The page shows `this.state.score`.
3. The parentheses call the function immediately while the JSX is being built, once per `Rating`. `onClick` receives its return value, `undefined`, so there is nothing to call on click.
4. React calls the handler as a plain function, not as a method of the component, so `this` is lost. An arrow function has no `this` of its own; as a class property, it captures the component instance permanently.
5. Updates can be batched, so both calls read the same old `this.state.score`. Use the function form: `this.setState(prevState => ({ score: prevState.score + 1 }))`.
6. Any three of the following. Props come from the parent, while state is owned by the component. Props are read-only to the component, while state changes through `setState()`. Props configure a component, while state remembers changes over time. One component's state is often passed to its children as props.
7. The Header, the list and the Footer all need it, and `App` is their closest common parent. Data can only be passed down, so it must live above everything that uses it.
8. `App` passes a function (`handleRemoveMovie`) down as a prop. `Movie` calls it with its id, and the function, which belongs to `App`, calls `App`'s `setState()`.

9. It can be calculated from `movies`. Storing it as well would create a second copy that must be kept in sync by hand. Computing it in render means it can never be wrong.
10. `push()` changes the existing array in place. State must be replaced with a new array, for example `[...prevState.movies, newMovie]`, so React can see that it changed.